



Towards Probabilistic Contracts for Intelligent Cyber-Physical Systems

Pauline Blohm¹ , Martin Fränzle² , Paula Herber¹ , Paul Kröger² ,
and Anne Remke¹

¹ University of Münster, Münster, Germany

{`pauline.blohm`,`paula.herber`,`anne.remke`}@uni-muenster.de

² Carl von Ossietzky Universität Oldenburg, Oldenburg, Germany

{`martin.fraenzle`,`paul.kroeger`}@uol.de

Abstract. Cyber-physical systems are systems that exhibit both discrete computational and continuous physical behavior. They are often subject to different kinds of uncertainty, ranging from sensor noise over random component failures to inconfidences induced by sample-based statistical learning. Quantitative formal methods have proven to be especially useful for assessing the impact of uncertainty on the system evolution over time. However, they lack compositionality. Existing methods for compositional design and verification, such as contracts, traditionally abstract from or (over-)approximate probability distributions, and resort to purely qualitative safety assessments in worst-case scenarios. This paper proposes a first step towards the integration of probabilistic methods into contract-based verification schemes to enable compositional reasoning over uncertain system behavior. We discuss different sources of uncertainties, as well as the necessity of probabilistic contracts for cyber-physical systems. Our key idea for integrating probabilities into contracts is the identification of safe yet precise approximations for sets of distributions, for which we use subdistributions. With that, we hope to reconcile probabilistic with set-based reasoning.

Keywords: Cyber-Physical Systems · Contracts · Quantitative Safety

1 Introduction

Cyber-physical systems (CPSes) are systems that integrate discrete computational components with a continuous physical environment, inducing complex hybrid discrete-continuous behavior emerging from their component combination. Such systems are used in a broad range of applications, and especially

The research reported herein originates from discussion at the Workshop on Contract Languages held at the Lorentz Center, Leiden, The Netherlands, March 4–8 2024. It has been partially funded by Germany’s Federal Ministry of Education and Research (BMBF) as part of AutoDevSafeOps (01IS22087Q) as well as by the Ministry of Science and Culture (MWK) of the State of Lower Saxony as part of Zukunftslabor Mobilität.

also for safety-critical applications in numerous domains such as the automotive industry, avionics, or energy supply. They inherently comprise various flavors of uncertainty affecting the system’s dynamic behavior. These uncertainties typically range from state and timing uncertainties, both on low and high abstraction levels, over uncertainties w.r.t. object or behavior classification and object existence to inherent epistemic uncertainties w.r.t., e.g., physical quantities.

Erroneous behavior of safety-critical systems can cause physical harm as well as high damage to property and reputation. A profound protection against undesirable system behavior is consequently highly desirable and often required by legal regulations. Under such circumstances, ignoring uncertainties is obviously an overly optimistic approach. On the other hand, covering them by non-deterministic choices may lead to overly pessimistic verification verdicts [5, 31]. Rather, uncertainties often have to be covered adequately in a quantitative manner upon verification of system compliance.

Formal methods in quantitative, statistical as well as purely qualitative variants are used to check the compliance of a system with its requirements with mathematical rigor. However, the systems under inspection are typically complex, i.e. compositions of sub-systems of various types and vendors. An overarching verification or simulation of the entire system is thus generally practically infeasible, since formal methods often suffer from the curse of dimensionality already for comparatively small systems. In addition, upon development of large systems, not all parameters of all system components are well-known for various reasons, in particular in early design phases. In case of reuse, a component may be applied in various different environments. Verification and validation of entire systems would thus usually be possible in very late development phases only. This means that behavioral system analysis would ultimately remain the responsibility of the OEM that assembles the overall system only—but who may not have the relevant internal information about the components.

The traditional divide-and-conquer approach in the form of contract-based design allows in principle both to tackle the curse of dimensionality and distribute the verification task over the component OEMs: The monolithic proof of the overall system is broken down into a combination of verification tasks for sub-systems and an integration of the resulting verdicts. In this context, a contract is formed by an assume-guarantee pair (A, G) , where an assumption A specifies a set of input traces while a guarantee G specifies a set of output traces. A system S satisfies a contract (A, G) if it guarantees a trace from G whenever the input is in A , i.e. $\llbracket S \rrbracket \cap A \subseteq G$, where $\llbracket S \rrbracket$ denotes the traces S may exhibit. Please note that we here, like in the behaviour-set based contract theory of Benveniste et al. [9] and Incer [43], consider traces to reside in a joint universe of traces that always mentions all ports of all system components, no matter whether a particular port belongs to the component under inspection or not. In the latter case, the corresponding port values remain unconstrained, i.e. the trace sets $\llbracket S \rrbracket$, A , and G are Cartesian products with the respective full domain sets. If now a system S consists of a serially composed chain $C_1 \succrightarrow C_2$ of two components C_i (with $i \in \{1, 2\}$) for which compliance with the corresponding

contracts (A_i, G_i) was proven, the goal of the compositional reasoning is then, in principle, to prove $A \subseteq A_1$, $G_1 \subseteq A_2$, and $G_2 \subseteq G$.

Integrating probabilistic methods into the contract-based design scheme—as safety-critical CPSes often would require—is by no means a trivial application of well-known probabilistic methods such as stochastic model-checking, as the compositional character of contract-based reasoning shall be preserved. Intuitively, one might go for characterizing a component’s input by a distribution over input traces in order to compute or estimate an output distribution.

Such an approach allows for compositional reasoning as long as input distributions are known exactly. This assumption however is inconsistent with the contract-based reasoning perspective: Knowing a component’s input distribution means to have exact and infallible information about the environmental behavior. This is usually not the case for various reasons such as distributed development of system components, protection of intellectual property, interchangeability or instantiation of the environment, or uncertainties prohibiting computation or estimation of dedicated distributions for the environmental behavior. In the contract-based sense, one should rather reason about sets of possible input and output distributions—just as the traditional contract-based method reasons about sets of possible traces rather than individual traces. Representing assumptions and guarantees as sets of possible distributions provides the liberty of applying the respective component in a variety of contexts—namely all scenarios matching one of the possible input distributions—as well as providing different refinements and implementations of the contract—namely all those mapping all the possible input distributions to one of the permissible output distributions. Manipulating sets of distributions however requires representation of such distribution sets, for which we suggest the upward closure of subdistributions.

In this paper, we 1) analyze the necessity of probabilistic contracts and the problems involved in establishing such contracts in the context of intelligent safety-critical CPSes with machine-learning components and 2) propose subdistribution as a means of characterization of sets of distributions as required for contract-based reasoning.

The paper is organized as follows. Section 2 provides a survey through existing modeling and evaluation approaches for safety-critical CPSes whereas Section 3 elaborates on sources of uncertainties in “intelligent” CPSes. We express the necessity of probabilistic contracts and formulate our proposal for a corresponding solution to this problem in Sect. 4, which is demonstrated on a small example in Sect. 5 before we conclude this paper with Sect. 6.

2 Existing Modeling and Evaluation Approaches

There exist several communities that work on problems related to the one described above. The program verification community has made huge progress towards interactive and automated tools for software verification. However, they usually do not consider hybrid or probabilistic systems. The embedded or

cyber-physical systems community has developed a broad range of model-based approaches to conquer the complexity of systems that encompass heterogeneous models of computation. However, these approaches typically either support quantitative aspects via (informal) simulation and testing, or focus on purely qualitative aspects using formal methods that enable analyses of embedded or cyber-physical systems that encompass both time-discrete and time-continuous behavior. Sophisticated modeling and analysis techniques such as timed automata, hybrid automata, symbolic state-space construction techniques and flowpipe construction are notable results of these efforts. However, these techniques typically neither support non-linear control systems nor probabilistic aspects. Finally, the probabilistic and stochastic modeling community, mainly centered around all aspects of Markov decision processes, has developed powerful tools and techniques to cope with stochastic processes and to incorporate probabilistic and quantitative aspects. However, they typically (only) consider time-bounded properties and provide statistical guarantees. In the following, we briefly summarize the main characteristics and achievements of each of these three communities, and reflect on the applicability of the developed tools and techniques for quantitative analyses of intelligent cyber-physical systems in the presence of uncertainties.

2.1 Contracts-Based Design and Verification

Within the program verification community, different kinds of contracts have been proposed and applied [60]. A classical form of contracts are pre-postcondition specifications, where assumptions on a piece of software or a program are captured using preconditions, and guarantees are captured as postconditions. Such pre-postcondition style contracts date back to the early 70-ies [40], and have been used to specify the behavior of procedures since the 90-ies, when Meyer introduced the idea of *Design by Contract* [54]. Other forms of contracts have been introduced, using temporal logics and automata, which are for example used to support component-based development of software systems [17]. While pre-postconditions usually specify the behavior in a state-pair fashion, temporal logics, dynamic logics, and automata also enable the specification of dynamic interaction behavior.

Contract-based design is today at the core of many interactive and automated tools for software and program verification, with significant progress made [6, 12, 28, 36, 57]. The underlying principle of assume-guarantee pairs is a powerful approach for compositional and modular reasoning. However, while extensive work exists for both functional and non-functional properties, probabilities and quantitative properties are typically not considered within this community.

In the cyber-physical systems community, probabilistic contracts however have been considered due to the need of dealing with uncertainties in environmental perception, unreliability of physical components, and only statistically characterized machine-learning components [23, 49, 65]. Being based on trace properties satisfied with (at least) a specified probability, these contract frameworks do however suffer from lacking practical applicability in compositional reasoning: with the assumption and guarantee being probabilistic state-

ments of the form “trace property ϕ holds with a probability of at least p ”, assume-guarantee pairs essentially talk about a large, often uncountable family of conditional probabilities which are very hard to verify. Verifying that a component C satisfies, e.g., the guarantee $G \equiv P(|output - input| < 0.1) > 0.9$ if the assumption $A \equiv P(0 \leq input \leq 1) > 0.95$ is met would require verifying satisfaction of G for any instance of A . In particular, one has to verify that $P(|output - input| < 0.1) > 0.9$ holds for any particular value of $input \in [0, 1]$, destroying the probabilistic character of the assumption. It consequently comes at no surprise that [49], while syntactically permitting probabilistic assumption, do not actually use them in any of their examples of contract reasoning, where they rather resort to qualitative assumptions.

2.2 Formal Verification of Hybrid Systems

There exists a broad body of work that aims at ensuring the safe behavior of cyber-physical or hybrid systems, ranging from systematic testing and runtime verification to model checking. Testing-based approaches, e.g. [18, 26], are industrially most successful and widely used for quality assurance of autonomous cyber-physical systems. However, they are inherently incomplete and monolithic, rendering them inapt to certify extremely high requirements of safety-critical complex systems. Runtime verification, e.g., [22, 55], complements early system verification, but usually requires a verified model to generate monitors or runtime conditions from. In addition, uncertainties also take hold on runtime monitors, deeming their results uncertain as well.

Approaches that use bounded or unbounded reachability computation provide guarantees that safety properties are satisfied in the reachable states of the system [30, 61]. These approaches typically rely on state-based formalisms such as hybrid automata, and perform reachability analysis based either on reach-set computation, for example, using interval constraint propagation [10, 45] or flow-pipe computations [16, 32] or on satisfiability solving for path formulae including ordinary differential equations [27]. Hybrid automata are finite state machines where in each discrete state, the evolution of continuous variables is described by a set of ordinary differential equations. Due to the underlying finite state formalism, these approaches generally assume that crucial system parameters, like the number of components or messages, are fixed. Furthermore, they typically do not scale well for larger systems, as they suffer from the state space explosion problem, i.e., the state space exponentially grows with the combination of concurrent processes and variables. To ease the verification process of concurrent systems, compositional verification approaches have been proposed, e.g. [10, 13]. These approaches enable the individual verification of components and build overall proofs from compositional verification results. For example, in [13], the authors integrate assume-guarantee reasoning into the model checker SpaceEx [32]. In [10], assume-guarantee reasoning is integrated in the verification framework Ariadne. However, while compositional verification reduces the verification effort, it neither resolves the restriction to fixed system parameters and finite state spaces

nor generally solves the state space explosion problem, and compositional reasoning techniques are not available for stochastic properties like quantitative safety targets.

In the last decade, deductive verification has become a powerful tool to overcome these limitations even for cyber-physical, hybrid systems, which comprise time-discrete and time-continuous behavior [29, 34, 56]. Platzer et al. have presented the differential dynamic logic ($d\mathcal{L}$) in [59]. $d\mathcal{L}$ is a logic for formally specifying and reasoning about properties of hybrid programs, which are a formal modeling language for hybrid systems. Hybrid programs describe the reachable states of a hybrid system using discrete assignments and continuous evolutions. With that, safety and liveness properties can be specified, and deductively verified with the interactive theorem prover KeYmaera X [56] using a generalized free-variable sequent calculus and automatic proof strategies [34]. Deductive verification provides safety guarantees even for parameterized systems under all circumstances, including infinite executions and arbitrary inputs. By exploiting inference rules and invariants, deductive verification enables inductive reasoning and thus avoids the state-space explosion problem. However, the definition of the necessary invariants and assertions requires high manual effort and expertise. In addition, compositionality is limited by the necessity to manually define the required invariants and contracts, as well as by the lack of a parallel composition operator in the hybrid programs of $d\mathcal{L}$. Also, quantitative properties are usually not considered.

2.3 Model-Based Evaluation in the Presence of Uncertainty

Closed cyber-physical systems, i.e. CPSes devoid of environmental inputs, with uncertainty are often modeled as stochastic hybrid models. These models extend the underlying model class of hybrid models with stochastic behavior in different ways. However, the scalability and accuracy of analysis techniques for this class of models is restricted by the undecidability of reachability for more expressive hybrid automata [38]. Real-time systems have successfully been modeled as timed automata [7, 39], where clocks evolve with rate one. The problem of *location reachability* of this restricted class is decidable [7]. While powerful tools like UPPAAL [8] are available for their evaluation and scheduling, scalability of analysis remains an issue, when data abstraction is not available, as in UPPAAL.

Realistic modeling of QoS in real-time systems requires incorporating uncertainty using probabilistic behavior. Probabilistic Timed Automata (PTA), as proposed in [47] extend timed automata with both probabilistic and nondeterministic parts. They have been used successfully to model communication protocols like ZigBee [33] and the AODV routing protocol [44]. Interval Probabilistic Timed Automata [66] were introduced as an extension of PTA and include uncertainty intervals for probabilities, which allows probabilities to change over time. Probabilistic real-time properties can then be expressed using Probabilistic Timed Computation Tree Logic, for which also a symbolic model checking approach exists (c.f. [66]). Note that, due to the inherent nondeterminism in PTAs, only minimum and maximum probabilities and expected values can be

computed. The model checking tools Prism [46] and Modest [37] support the analysis of PTA. The lack of scalability can be circumvented by statistical model checking, which is available for PTAs [21] and provides (statistical) guarantees to quantitative metrics [48].

More general classes of stochastic hybrid models (SHM) exist, which allow for more complex behavior, e.g. including continuous evolution described by (stochastic) differential equations [3, 52]. As witnessed by the regular ARCH competition [1, 2], models with more complex behavior are intrinsically difficult to analyze. Different formalisms focus on different modeling features. Some exclude nondeterminism such that all decisions are taken randomly [11, 14, 15]. The resulting models are then stochastic processes in continuous or discrete time. Other approaches [24, 62] extend hybrid automata with stochasticity. For an overview on how different work incorporates stochasticity into hybrid automata, we refer to [64].

An additional difficulty lies in the lack of compositional modeling and analysis methods for SHM, as most existing approaches systems are monolithic. Notable exceptions are [19, 35] and [63].

Approaches based on statistical simulations, e.g., [20, 58], provide quantitative statistical guarantees and can also be used for performance evaluation. However, they perform a monolithic evaluation of the overall system, and can not easily be applied component-wise, as this would require to consider open systems where some input distributions might not be known (exactly).

3 Sources of Uncertainties in Intelligent CPSes

In intelligent cyber-physical systems, there exist several sources of uncertainty. In this section, we discuss kinds of uncertainties and their main characteristics based on a case study of an adaptive cruise control (ACC) system provided by Mathworks [53]. Mathworks provides this example to demonstrate the use of the reinforcement learning (RL) toolbox. The case study has also been used in [4], where we have presented an approach for reusable contracts that describe learning components in intelligent hybrid systems in differential dynamic logic (d $\mathcal{L}$). The ACC case study, though simple, is a typical intelligent cyber-physical system in the sense that a discrete controller implementing a learning algorithm is embedded into a physical environment and interacts with this environment via sensors and actuators. While the discrete controller, i.e. the learning component, takes control decisions at discrete sample times, the technical environment, i.e. the physical behavior of the follower and the lead car, evolves continuously according to differential equations.

Adaptive Cruise Control System. The structure of the ACC system is shown in Fig. 1. It consists of two main components: a lead car and a follower car with adaptive cruise control (ACC). The lead car models the dynamic environment of the follower car. The follower car with ACC observes the position p_L and speed (velocity v_L) of the lead car with a signal processing unit, which may

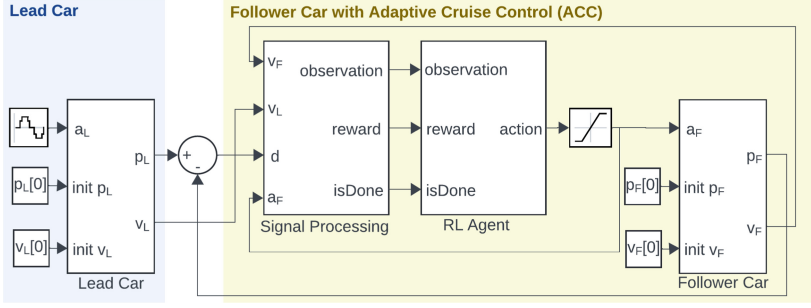


Fig. 1. Adaptive Cruise Control System, adopted from [53].

contain simple distance sensors or more complex image processing capabilities. At the core of the follower car with ACC, a reinforcement learning (RL) agent observes the distance $d = p_L - p_F$ to the lead car and its velocity v_L as well as the current velocity v_F and acceleration a_F of the follower car itself. The signal processing unit additionally provides a reward to the RL agent, which is highest if a desired target velocity v_{ref} is reached while maintaining a safe distance from the lead car. The reward function is defined as $r = -(0.1 \cdot (v_{ref} - v_F)^2 + a_F^2) + M$, where M is a logical value that is set to 1 whenever $(v_{ref} - v_F)^2 \leq 0.25$ to give a positive reward for being close to the desired target velocity. The RL agent controls the ACC by choosing longitudinal acceleration and braking. The follower car component computes the resulting position p_F and velocity v_F by integrating the chosen acceleration a_F over time.

Sources of Uncertainty. There are several sources of uncertainty within the ACC case study:

- The sensors within the signal processing unit may yield uncertain results due to measurement errors, and sensors may fail at uncertain points in time.
- The signal processing unit may also yield massive mismasurements due to uncertainties concerning the identification of the lead car (i.e., another object may occasionally be mistakenly identified as the lead car).
- The RL agent uses randomization techniques, for example by randomly choosing between exploration and exploitation.
- The RL agent employs statistical learning from a finite set of samples, thus providing statistical guarantees only.
- The implementations of both the signal processing unit and the RL agent are subject to several uncertainties, ranging from floating point errors over failed memory accesses and bit flips to clock errors, which result in a skewed sample time, i.e. the period where the RL agent is executed may be reduced or increased.
- The effect of chosen control values, e.g. the chosen acceleration, is not necessarily the same as the acceleration the car actually drives with; this may

be due to uncertain actuators and uncertain physical effects such as wheel slip and friction, but also due to uncertain road conditions, and of course the general tracking error of the controller.

Kinds of Uncertainties. As these uncertainties are inherently present in cyber-physical systems, this clearly motivates the necessity to systematically capture them and assess their impact on the dynamic system behavior. To better understand the causes and effects of uncertainties, the authors of [25, 42] define *aleatoric* and *epistemic* uncertainty. Aleatoric or statistical uncertainty refers to inherently random effects, e.g. induced by coin flipping. In contrast, epistemic or systematic uncertainty refers to uncertainty caused by a lack of knowledge, e.g. about the mass of the car and its resulting inertia. While these concepts are useful to distinguish between uncertainties that may be reducible (epistemic) or irreducible because they result from real random effects (aleatoric), it is often difficult to clearly categorize a given uncertainty along this line. Often, it depends on the model we look at: for example, sensor noise looks purely random, but if we had a full model of environmental influences on the sensor, the uncertainty could be reduced significantly. Sometimes, the distinction however is clear and becomes visible within standard architectures for fault-tolerant systems: While memory corruption by radiation clearly is aleatoric, the uncertainty in predicting the behavior between actual test points of a neural net that is not employing internal randomization is purely epistemic—the net’s behavior is a deterministic function and thus defined, we just do not know it between test points. When we embed multiple copies of the two items into a redundant architecture, like triple modular redundancy, the difference becomes evident from the actual dynamic behavior: For the neural net, the voter will never be able to discover aberrant behavior, as there is none. Unless hit by a random fault, the three copies of the neural net will always generate equal output—whether right or wrong remains indistinguishable to the voter. Upon memory corruption, voting however is effective, showing that fundamentally different dynamics emerges from aleatoric and epistemic uncertainty in fault-tolerant architectures like redundancy plus majority.

Besides aleatoric and epistemic, we can distinguish measurement uncertainties, uncertainties about the current system state, classification uncertainties, existential uncertainties or uncertainties that are used for specification, for describing the behavior of something physical, or for defining algorithms that use fuzzy techniques or randomization. Furthermore, we can distinguish between uncertainties on the input/assumption or on the output/guarantee side of a given component. Modeling of uncertainties may also be used to replace unknown or non-deterministic behavior or to capture systematic faults.

Qualitative Specifications for the ACC Example. If we want to analyze and verify the ACC example in a hierarchical fashion using contracts for each component, we can write a contract C_{ACC} for the whole ACC that makes use

of the sub-contracts for the RL agent (C_{RL}), the braking system (C_B), and the signal processing unit (C_S).

In [4], we have presented a qualitative specification for the ACC case study in the differential dynamic logic (dL) [59], which provides purely qualitative contracts for the components. In particular, we have defined a contract for the RL agent that ensures that the RL agent always chooses safe actions as follows:

$$\begin{aligned} C_{RL} &= (A_{RL}, G_{RL}) \text{ with} \\ A_{RL} &= b > 0 \wedge p_F < p_L - \frac{v_F^2}{2 \cdot b}, \\ G_{RL} &= p_L - p_F - \left(v_F \cdot t_s + \frac{a_F \cdot t_s^2}{2} \right) > d_{min} + \frac{(v_F + a_F \cdot t_s)^2}{2 \cdot b}. \end{aligned} \quad (1)$$

If the RL agent chooses the acceleration of the follower car with ACC a_F according to this contract, it will always maintain a minimal distance plus the braking distance (computed from the current velocity v_F , acceleration a_F , braking force b , and sample time t_s) until the next discrete control decision. In [4], we have replaced the RL agent with this contract. Please note that Eq. 1 defines a contract predicate as a pre-postcondition pair. To ensure that it is correctly executed at each discrete sample time, we have additionally provided an embedding into the dL model in [4]. Then, we have deductively verified the resulting model using the interactive theorem prover KeYmaera X [34], in particular that the overall system always maintains a safe distance, as long as the RL agent chooses safe actions according to its contract. That is, we have verified the following system property for collision freedom:

$$[\alpha] p_L - p_F > d_{min} , \quad (2)$$

where α is a hybrid program we have automatically generated from the Simulink model using a previously proposed transformation from Simulink into the differential dynamic logic (dL) [50, 51], and $[\cdot]$ is the dL box modality, i.e. it expresses that the property should hold in all the reachable states of the given hybrid program α . The contract is embedded into α . Note that we have not verified the RL agent itself, but generated a runtime monitor from the contract, which is used as a shield for the RL agent to ensure safe actions at runtime.

While we have abstracted from measurement uncertainties in [4], the approach can easily be extended with assumptions on the measurement accuracy as a worst-case consideration. For example, the sensor accuracy could be qualitatively specified as follows:

$$|p^{nominal} - p^{measured}| \leq 0.75 \quad (3)$$

However, for such a qualitative specification, we always have to safely over-approximate, and thus will always end up with a worst-case consideration. This means that, if we do not have a guaranteed and tight enough bound on sensor errors, we will not be able to prove anything. Even worse, we will not be

able to prove anything about components that we can only characterize statistically by test or Monte-Carlo simulation, like computer-vision components. Even if qualitative verification succeeds, we often can get more precise information about failure rates, repair times, and error distributions, and it would be highly desirable to incorporate this knowledge into (quantitative) analysis techniques. The analysis and evaluation techniques presented in Sect. 2.3 provide us with means to assess the influence of failure rates and error distributions on the system evolution using, for example, statistical model checking. However, these techniques follow a monolithic approach, and do not provide us with contract-based, compositional reasoning techniques.

4 Towards Probabilistic Contracts

The problem with the aforementioned analysis techniques is that in a setting plagued by uncertainty, the component-based arguments do not really meet to give an overarching correctness statement: assume a (sub-)system of two serially composed synchronous components C_1 and C_2 that exchange information through channels from C_1 's output to C_2 's input such that their individual behavior in the actual application context can only be understood when knowing the other component's behavior. The components **Signal Processing** and **RL Agent** in Fig. 1 provide an example of such an architecture. The classical notion of an assume-guarantee contract for cyber-physical components [9] allows decomposing the *qualitative* correctness argument of the compound system by

1. formulating assume-guarantee pairs (A_i, G_i) for each component, where the A_i and G_i denote sets of behaviors (often in terms of traces of the respective components and specified by some linear-time temporal logic) such that, first, the A_i constrain the inputs to the corresponding components C_i , yet not their outputs, and that $G_i = G_i \cup \overline{A_i}$, i.e. that G_i , while allowed to relate outputs to inputs, does not constrain the input-output relation for inputs outside A_i ,
2. for each component C_i showing that component C_i satisfies contract (A_i, G_i) in that $\llbracket C_i \rrbracket \cap A_i \subseteq G_i$, i.e. C_i when subjected to input in A_i generates input-output traces that always satisfy the guarantee G_i ($\llbracket C_i \rrbracket$ here denotes the set of behaviors of component C_i),
3. finally, showing that these individual contracts (A_i, G_i) together imply satisfaction of the overall system specification (A, G) by proving $A \subseteq A_1, G_1 \subseteq A_2$, and $G_2 \subseteq G$. Note that none of these properties refers to the component implementations C_i any longer, but is merely based on their contracts (A_i, G_i) serving as their behavioral summary.

Contract-based reasoning thus establishes an overarching correctness argument for the total system without requiring monolithic verification of the whole system—in fact, completely different forms of proof can be used for justifying the various inclusions above within a compositional proof.

The problem with this type of reasoning in practice is that it requires infallible arguments about component behavior: it has, e.g., to be certified with 100% assurance that $\llbracket C_i \rrbracket \cap A_i \subseteq G_i$. Trying to obtain such certificates for cyber-physical components would, however, be elusive in general: while such statements may pertain to the nominal behavior of engineered software components, they fail to hold even for these components when the inevitable presence of (maybe rare) random failures in their runtime environment is acknowledged. For machine-learned components, statistical performance evaluation is state of the art, such that their pertinent reliability figures generally stay (considerably) below 100% and additionally are plagued by statistical (in-)confidences due to the certificates being based on finite, and thereby non-exhaustive, sample sets. And for inherently random components, like sensors with their noisy measurements, reliability also just is statistical at a reliability level below 100%.

For establishing compositional reasoning for realistic cyber-physical systems, we consequently are in urgent need of probabilistic variants of contracts. From our perspective, such a probabilistic contract framework ought to reconcile the following properties:

1. it permits compositional reasoning in a probabilistic setting, where probabilistic contracts for components provide summaries of the probabilistic —i.e., not 100% reliable— behavior of components that
 - (a) can be checked locally per component and
 - (b) suffice for rigorously inferring well-behavior at system level without recurrence to component designs, but just their contracts;
2. it supports component reuse in that it offers a rigorous notion of contract refinement or, equivalently, strengthening¹ such that a component fit for one contractually defined application context can be lifted to different contexts without additional component verification, namely to all contexts defined by weaker contracts;
3. it facilitates flexible integration, in arbitrary combination, of diverse types of evidence for the (sufficiently reliable) well-behavior of components, especially
 - (a) formal verification, like manual or interactive proof or automatic model-checking, establishing infallible correctness arguments for some components, combined with
 - (b) statistical evidence from testing, like proven-in-use arguments, randomized component tests, or statistical model-checking;
4. it is applicable to real systems in that its instantiation does not require procedures or characterizations of items that are impractical, like exact characterization of relative frequencies of use cases or of inputs to open systems.

Establishing such a framework, however, is hard, as we will show next by analyzing a seemingly natural variant.

¹ In the qualitative setting, a contract (A', G') is stronger than contract (A, G) iff $A \subseteq A'$ and $G' \subseteq G$, i.e. if it weakens assumptions and strengthens guarantees. In this case, any component satisfying (A', G') can also safely be used in a context requiring contract (A, G) .

4.1 Employing Behavior Distributions Instead of Behavior Sets as Assumptions and Guarantees

In order to obtain a probabilistic notion of contract, it would seem natural to replace the sets of behaviors A_i and G_i by distributions over behaviors. To this end, let $Behav$ be the set of possible output behaviors, normally described in terms of subsets of both the input and output variables of the component C_i .² Correspondingly, let $IBehav$ be the set of possible input behaviors to the component C_i , which may only be described in terms of input variables. Now let $A_i \in \mathcal{D}(IBehav)$ and $G_i \in \mathcal{D}(Behav)$ be distributions over input behaviors and input-to-output behaviors of C_i , respectively. Let us denote by $\llbracket C_i \rrbracket(A) \in \mathcal{D}(Behav)$ the distribution of input-to-output behaviors arising when applying the component C_i to inputs distributed according to distribution A .

Then, in absence of an inclusion relation (or an equivalent domination order) for distributions resembling that for sets of behaviors, the only natural definition or satisfaction of the contract (A, G) by the component C_i would be

$$\llbracket C_i \rrbracket(A_i) = G_i . \quad (4)$$

This notion, while offering the right shape for transitive reasoning across components, would nevertheless fail in many respects.

First, it would require prior knowledge of the exact application context, as the distribution A_i mentioned in the contract would have to exactly match the application context, because Eq. 4 does not imply anything about C_i 's behavior on any other—even an approximately equal—input distribution. The above properties 1 and 4 required of a reasonable contract framework are consequently not met. Second, Eq. 4 precludes any reuse of proven components in other contexts or any refinement of contracts, again due to the need of exact match between distributions. This contradicts required property 2 from above. Third, we would not be able to employ statistical model-checking or any other empirical exploration techniques, as required by property 4, to demonstrate that a component satisfies Eq. 4: Empirical methods can only approximately characterize distributions, thus being inapt to generate the verdicts that an exact distribution match as in Eq. 4 would require.

² “In terms of input and output variables” is to be understood broadly here. There is no need to express component behaviors directly in terms of the immediate (input and output) port variables of the component, as the mathematical framework applies equally well to images thereof obtained by coordinate transformations and projections or even non-linear images of the state-space spanned by the ports, as long as the semantics $\llbracket C \rrbracket$ of a component is well-defined over these images. In the example contract provided in Sect. 5, we will describe the input-output behavior by a one-dimensional (sub-)distribution on the difference $d = a_{control} - \frac{1}{2} \frac{v^2}{s}$ between physical states and the value of a control output, rather than by a joint distribution on all these variables.

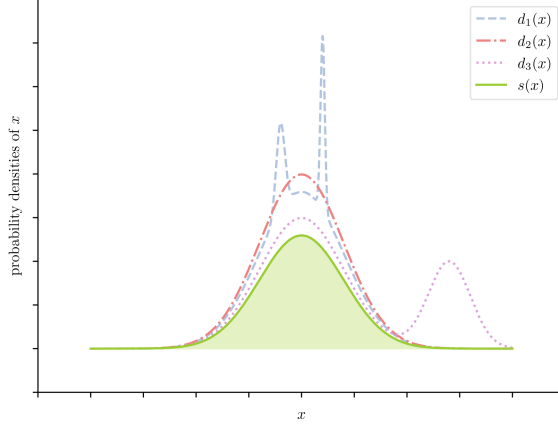


Fig. 2. An example of a subdistribution s that is dominated by (a.o.) d_1 , d_2 , and d_3 .

4.2 Subdistributions in Assumptions and Guarantees

It seems that most, if not all, of the aforementioned problems can be alleviated by using subdistributions instead of distributions for capturing assumptions and guarantees. A subdistribution P over a measurable space (M, Σ) , where M is the underlying set and Σ the σ -algebra over it, is a measure $P : \Sigma \rightarrow [0, 1]$ satisfying $P(M) \leq 1$, rather than the equality $P(M) = 1$ required for distributions. Note that being a measure with co-domain $[0, 1]$, it otherwise shares all properties of probability distributions. We denote the set of subdistributions over a measurable space (M, Σ) by $\mathcal{SD}(M, \Sigma)$ or, whenever the underlying σ -algebra Σ is understood, by $\mathcal{SD}(M)$.

In contrast to distributions, subdistributions feature a natural partial ordering: we say that subdistribution $s' \in \mathcal{SD}(M)$ dominates subdistribution $s \in \mathcal{SD}(M)$, denoted $s' \sqsupseteq s$, iff $\forall \sigma \in \Sigma : s'(\sigma) \geq s(\sigma)$. Distributions are maximal subdistributions wrt. $\sqsupseteq$. Therefore, any subdistribution $s \in \mathcal{SD}(M)$ represents a set of distributions that we denote by $s^\uparrow$, namely $s^\uparrow = \{d \in \mathcal{D}(M) \mid d \sqsupseteq s\}$. Consequently, $s' \sqsupseteq s$ implies $s'^\uparrow \subseteq s^\uparrow$. Figure 2 illustrates an example of a subdistribution s in terms of a probability density function $s(x)$ with $\int s(x)dx < 1$ that is dominated by (a.o.) $d_1, d_2, d_3 \in s^\uparrow$, i.e. $s \sqsubseteq d_i$ with $i \in \{1, 2, 3\}$. More precisely, in this example $s^\uparrow$ is the set of all distributions $d \in \mathcal{D}(\mathbb{R})$ satisfying $\int_a^b d(x)dx \geq \int_a^b s(x)dx$ for all real-valued intervals $[a, b]$ with $a < b \in \mathbb{R}$.

Subdistributions thus reconcile probabilistic reasoning with set-based reasoning, with the latter being at the heart of compositional proof schemes based on contracts. We consequently suggest employing subdistributions over $IBehav$ and over $Behav$ as assumptions and guarantees, respectively. The natural notion of satisfaction of a contract (A, G) , with $A \in \mathcal{SD}(IBehav)$ and $G \in \mathcal{SD}(Behav)$, by a component C then is

$$\forall d \in A^\uparrow : \llbracket C \rrbracket(d) \in G^\uparrow . \quad (5)$$

It is natural to extend the semantics $\llbracket C \rrbracket$ to subdistributions. The canonical generalization of the probabilistic semantics $\llbracket C \rrbracket$ to subdistributions s satisfies

$$\llbracket C \rrbracket(s) = s(M) \cdot \llbracket C \rrbracket \left(\frac{1}{s(M)} \cdot s \right), \quad (6)$$

where multiplication of (sub-)distributions with scalars is pointwise as in $(\alpha \cdot s)(\sigma) = s(\sigma) \cdot \alpha$ for all $\sigma \in \Sigma$. Consequently, Eq. 5 is equivalent to

$$\llbracket C \rrbracket(A) \supseteq G \quad (7)$$

due to the monotonicity of $\llbracket C \rrbracket$ trivially implied by the pointwise operation of C on input behaviors and the fact that $\llbracket C \rrbracket$ maps distributions to distributions.

Because of the correspondence between subdistribution ordering and set inclusion on the implied distributions, refinement now works as for qualitative contracts: if C satisfies (A, G) according to Eq. 7 and $A' \supseteq A$ and $G \supseteq G'$ then C also satisfies (A', G') . We thus say that (A, G) is stronger than, i.e. refines (A', G') iff $A' \supseteq A$ and $G \supseteq G'$.

4.3 Normalized Subdistribution Contracts

We observe the following straightforward linearity property of stochastic system semantics over subdistributions:

Lemma 1. (Linearity). *Let C be a component, $d \in \mathcal{SD}(\text{IBehav})$ and $\alpha \in [0, 1]$. Then $\llbracket C \rrbracket(\alpha d) = \alpha \llbracket C \rrbracket(d)$ holds.*

Proof. The statement follows immediately from Eq. 6. $\square$

This lemma permits reducing subdistribution contracts to a normalized form, where assumptions are distributions rather than subdistributions, as any subdistribution can be obtained by scaling a distribution.

A *normalized subdistribution contract* (A, G) therefore is a pair of a distribution $A \in \mathcal{D}(\text{IBehav})$ and a subdistribution $G \in \mathcal{SD}(\text{Behav})$. As in the general subdistribution case, cf. Eq. 7, we say that component C satisfies the contract (A, G) iff $\llbracket C \rrbracket(A) \supseteq G$. Note that due to the linearity Lemma 1, this implies that C also satisfies all the subdistribution contracts $(\alpha A, \alpha G)$ for any $\alpha \in [0, 1]$, i.e. that (A, G) can serve as a normalized representation of that family of contracts.

This normalized contract, however, has the advantage that it can be checked by statistical model-checking and similar empirical means, as the distribution on the input side, as well as the component's reaction to such inputs, can be sampled. The resulting empirical input-to-output distribution suffices for approximating an input-to-output subdistribution with confidence:

Lemma 2. (Statistical contract refutation (SCR)). *Let $i_1, \dots, i_N$ be a family of random samples from IBehav independently and identically drawn according to distribution A and let $r_1, \dots, r_N$ be the reactions obtained from simulating (in i.i.d. fashion if C internally is stochastic) the component C over*

these inputs. Then C violates the normalized subdistribution contract (A, G) with confidence $1 - e^{-\frac{2t^2}{N}}$ if

$$\exists \sigma \in \Sigma : G(\sigma) > \frac{|\{j \in \{1, \dots, N\} \mid r_j \in \sigma\}|}{N} + t . \quad (8)$$

Proof. Let $\sigma \in \Sigma$ and consider the random variable $x = \chi(\llbracket C \rrbracket(i) \in \sigma)$, where χ maps true to 1 and false to 0, as usual. Computing $\frac{|\{j \in \{1, \dots, N\} \mid r_j \in \sigma\}|}{N}$ provides an empirical estimate $\tilde{\mathcal{E}}_x$ for the expected value $\mathcal{E}_x$ of x over input distribution A , i.e. when inputs i are drawn according to the distribution A . According to Hoeffding's inequality [41], the likelihood $P(\mathcal{E}_x - \tilde{\mathcal{E}}_x) > t$ of empirically underestimating $\mathcal{E}$ by more than t satisfies $P(\mathcal{E}_x - \tilde{\mathcal{E}}_x > t) \leq e^{-\frac{2t^2}{N}}$, which proves the statement. $\square$

Note that requirement 3(b) demanded of a reasonable probabilistic contract framework consequently is met. As qualitative evidence from exhaustive rigorous proofs, like manual or interactive proofs or automatic model-checking, trivially dominates any corresponding subdistribution, the seamless integration of formal verification required in 3(a) also is possible. Requirement 3 consequently is met.

Remark 1. It might seem attractive to also do positive statistical model-checking of components C against normalized contracts (A, G) , rather than a refutation, by checking for satisfaction of

$$\forall \sigma \in \Sigma : G(\sigma) \leq \max \left(\frac{|\{j \in \{1, \dots, N\} \mid r_j \in \sigma\}|}{N} - t, 0 \right) .$$

While this criterion is sound, it unfortunately is useless, as only the trivial subdistribution $G \equiv 0$ satisfies this criterion. The reason is the additivity of the measure G , together with the fact that the right-hand side of the inequality inevitably becomes 0 for almost all small sets $\sigma \in \Sigma$.

Relaxing the additivity requirement for subdistributions would, however, invalidate the transitivity inherent to compositional contract reasoning, which for additive measures follows underneath. $\square$

The notion of normalized subdistribution contract allows for transitive contract reasoning as follows:

Lemma 3. (Compositional reasoning). *Let C_1 and C_2 be two serially composed components and let the two C_i satisfy their mutual normalized subdistribution contracts (A_I, G_I) .*

Then their composition $C_1 \succcurlyeq C_2$ satisfies $(A_1, \alpha G_2)$ for any factor α satisfying $G_1|_{IBehav_2} \sqsupseteq \alpha A_2$, where $G_1|_{IBehav_2}$ is G_1 marginalized to the input behaviors $IBehav_2$ of C_2 .

Proof. Component C_1 satisfies the contract (A_1, G_1) , which by the restated definition of contract satisfaction provided by Eq. 5 implies $\llbracket C_1 \rrbracket(A_1) \sqsupseteq$

G_1 . The composition $C_1 \succrightarrow C_2$ then satisfies $\llbracket C_1 \succrightarrow C_2 \rrbracket(A_1) = \llbracket C_2 \rrbracket((\llbracket C_1 \rrbracket(A_1)|_{I\text{Behav}_2}) \sqsupseteq \llbracket C_2 \rrbracket(G_1|_{I\text{Behav}_2}))$ by monotonicity of $\llbracket C_2 \rrbracket$.

If $G_1|_{I\text{Behav}_2} \sqsupseteq \alpha A_2$ then, again by monotonicity and linearity of $\llbracket C_2 \rrbracket$ and by satisfaction $\llbracket C_2 \rrbracket(A_2) \sqsupseteq G_2$ of contract (A_2, G_2) by C_2 , it follows that $\llbracket C_1 \succrightarrow C_2 \rrbracket(A_1) \sqsupseteq \llbracket C_2 \rrbracket(G_1|_{I\text{Behav}_2}) \sqsupseteq \llbracket C_2 \rrbracket(\alpha A_2) \sqsupseteq \alpha G_2$. $\square$

Note that this lemma implies that the subdistribution contract framework meets property 1 from the requirements we posed on a reasonable probabilistic contract framework.

With properties 1–3 being met, it only remains to be shown that subdistribution contracts are applicable to real systems in that their instantiation does not require procedures or characterizations of items that are impractical (i.e., requirement 4 on a reasonable contract framework). We demonstrate this in the following section by means of an example.

5 An Example of a Subdistribution Contract

A natural specification of a control process under uncertainty in the vein of the ACC example (Sect. 3) would state that whenever the measured quantities are probable to be close to the real values, then the control output is probable to be adequate. Using the notation $D(x)$ for the distribution of a random variable x (after marginalization to x), this can be stated as follows:

Assumption: Statement $D(d - d_{\text{measured}}) \sqsupseteq \max(0.98 \cdot \mathcal{N}(0\text{m}, 1\text{m}) - 0.02, 0) \wedge D(v - v_{\text{measured}}) \sqsupseteq \max(0.99 \cdot \mathcal{N}(0\frac{\text{m}}{\text{s}}, 0.5\frac{\text{m}}{\text{s}}) - 0.01, 0)$, where $\mathcal{N}(x, \mu)$ denotes the normal distribution around x with variance μ , states that the measurement error for the distance d to car in front (and car velocity v , resp.) features a distribution that dominates a subdistribution given by a scaled and down-shifted normal distribution. Note that in contrast to the frequently encountered assumption of normally distributed measurement error, which is mathematically convenient but physically unrealistic, this subdistribution specification permits different realizations and even conditional variations of the distribution depending on the current physical velocity value v , as necessary for admitting e.g. quantization by analog-digital conversion.

Guarantee: $D(a_{\text{control}} - \frac{1}{2} \frac{v^2}{d}) \sqsupseteq 0.9 \cdot \mathcal{U}(-1\frac{\text{m}}{\text{s}^2}, 0\frac{\text{m}}{\text{s}^2})$, where $\mathcal{U}(l, u)$ is the uniform distribution between l and u , states that the acceleration value a_{control} suggested by the control algorithm will, with high likelihood, (a) not exceed the physically required maximal value $\frac{1}{2} \frac{v^2}{d}$ and (b) stay close to the required value by a margin of $1\frac{\text{m}}{\text{s}^2}$.

Please note that $D(a_{\text{control}} - \frac{1}{2} \frac{v^2}{d})$ talks about the distribution of the difference $a_{\text{control}} - \frac{1}{2} \frac{v^2}{d}$ such that it well permits stochastic dependencies between the physical ground-truth value $\frac{1}{2} \frac{v^2}{d}$ and the control value a_{control} , like quantization of the latter.

6 Conclusion

In this paper, we elaborated on the necessity of probabilistic contracts in contract-based reasoning for intelligent safety-critical CPSes which typically incorporate various flavors of uncertainty. As soon as the environment of a component is not exactly known, which is often the case in development of large, complex systems, classical probabilistic methods fail to provide a compositional means of verification. The reason is that a set-based reasoning over input and output traces, as known from traditional contract-based reasoning, is required. A meaningful quantitative verification of system compliance would thus have to deal with sets of distributions over input and output traces. In this paper, we suggest subdistributions as a means of describing sets of such distributions. We think that subdistributions have the potential to reconcile probabilistic and set-based reasoning, and thus provide a first step towards probabilistic contracts.

In future work, we plan to extend our work with a larger case study to investigate whether subdistributions actually provide us with a feasible and meaningful approach for assume-guarantee reasoning over complex intelligent hybrid systems. Furthermore, if this first step is successful, we plan to develop a specification language for probabilistic contracts based on subdistributions.

References

1. Abate, A., et al.: ARCH-COMP23 category report: stochastic models. In: Proceedings of 10th International Workshop on Applied Verification of Continuous and Hybrid Systems. EPiC Series in Computing, vol. 96, pp. 126–150. EasyChair (2023). <https://doi.org/10.29007/k7s6>
2. Abate, A., et al.: ARCH-COMP22 category report: stochastic models. In: Proceedings of 9th International Workshop on Applied Verification of Continuous and Hybrid Systems. EPiC Series in Computing, vol. 90, pp. 113–141. EasyChair (2022). <https://doi.org/10.29007/lsvc>
3. Abate, A., Katoen, J.P., Mereacre, A.: Quantitative automata model checking of autonomous stochastic hybrid systems. In: Proceedings of the 14th international conference on Hybrid Systems: Computation and Control, pp. 83–92. ACM (2011). <https://doi.org/10.1145/1967701.1967715>
4. Adelt, J., Brettschneider, D., Herber, P.: Reusable contracts for safe integration of reinforcement learning in hybrid systems. In: Automation Techniques for Verification and Analysis. Springer (2022). https://doi.org/10.1007/978-3-031-19992-9_4
5. Adelt, J., Herber, P., Niehage, M., Remke, A.: Towards safe and resilient hybrid systems in the presence of learning and uncertainty. In: 11th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation, pp. 299–319. Springer (2022). https://doi.org/10.1007/978-3-031-19849-6_18
6. Ahrendt, W., et al.: The KeY platform for verification and analysis of java programs. In: Verified Software: Theories, Tools, and Experiments, pp. 1–17, No. 8471 in LNCS, Springer (2014). https://doi.org/10.1007/978-3-319-12154-3_4
7. Alur, R., Dill, D.L.: A theory of timed automata. Theoret. Comput. Sci. **126**, 183–235 (1994). [https://doi.org/10.1016/0304-3975\(94\)90010-8](https://doi.org/10.1016/0304-3975(94)90010-8)

8. Behrmann, G., David, A., Larsen, K.G.: A Tutorial on Uppaal. In: Formal Methods for the Design of Real-Time Systems, LNCS, vol. 3185, pp. 200–236. Springer (2004). https://doi.org/10.1007/978-3-540-30080-9_7
9. Benveniste, A., et al.: Contracts for system design. *Found. Trends Electron. Des. Autom.* **12**(2–3), 124–400 (2018). <https://doi.org/10.1561/10000000053>
10. Benvenuti, L., Bresolin, D., Collins, P., Ferrari, A., Geretti, L., Villa, T.: Assume–guarantee verification of nonlinear hybrid systems with Ariadne. *Int. J. Robust Nonlinear Control* **24**(4), 699–724 (2014). <https://doi.org/10.1002/rnc.2914>
11. Bertrand, N., et al.: Stochastic timed automata. *Logical Methods Comput. Sci.* **10**(4) (2014). [https://doi.org/10.2168/lmcs-10\(4:6\)2014](https://doi.org/10.2168/lmcs-10(4:6)2014)
12. Blom, S., Huisman, M.: The VerCors tool for verification of concurrent programs. In: International Symposium on Formal Methods, pp. 127–131. Springer (2014). https://doi.org/10.1007/978-3-319-06410-9_9
13. Bogomolov, S., et al.: Assume-guarantee abstraction refinement meets hybrid systems. In: Haifa Verification Conference, pp. 116–131. Springer (2014). https://doi.org/10.1007/978-3-319-13338-6_10
14. Bujorianu, M.L., Lygeros, J.: Toward a General Theory of Stochastic Hybrid Systems, pp. 3–30. Springer (2006). https://doi.org/10.1007/11587392_1
15. Cauchi, N., Abate, A.: StocHy: automated verification and synthesis of stochastic processes. In: Tools and Algorithms for the Construction and Analysis of Systems, pp. 247–264. Springer (2019). <https://doi.org/10.1145/3302504.3313349>
16. Chen, X., Ábrahám, E., Sankaranarayanan, S.: Flow*: an analyzer for non-linear hybrid systems. In: International Conference on Computer Aided Verification, pp. 258–263. Springer (2013). https://doi.org/10.1007/978-3-642-39799-8_18
17. Cimatti, A., Dorigatti, M., Tonetta, S.: OCRA: a tool for checking the refinement of temporal contracts. In: 28th IEEE/ACM International Conference on Automated Software Engineering, pp. 702–705. IEEE (2013). <https://doi.org/10.1109/ASE.2013.6693137>
18. Dang, T., Nahhal, T.: Coverage-guided test generation for continuous and hybrid systems. *Formal Methods Syst. Des.* **34**(2), 183–213 (2009). <https://doi.org/10.1007/s10703-009-0066-0>
19. David, A., Du, D., Kim G. Larsen, A.L., Mikučionis, M., Poulsen, D.B., Sedwards, S.: Statistical model checking for stochastic hybrid systems. In: Electronic Proceedings in Theoretical Computer Science, vol. 92, pp. 122–136 (2012). <https://doi.org/10.4204/EPTCS.92.9>
20. David, A., Larsen, K.G., Legay, A., Mikučionis, M., Wang, Z.: Time for statistical model checking of real-time systems. In: Computer Aided Verification, pp. 349–355. Springer (2011). https://doi.org/10.1007/978-3-642-22110-1_27
21. David, A., Larsen, K.G., Legay, A., Mikučionis, M., Poulsen, D.B.: UPPAAL SMC tutorial. *Int. J. Softw. Tools Technol. Transf.* **17**(4), 397–415 (2015). <https://doi.org/10.1007/s10009-014-0361-y>
22. Decker, N., Leucker, M., Thoma, D.: Monitoring modulo theories. *Int. J. Softw. Tools Technol. Transfer* **18**(2), 205–225 (2016). <https://doi.org/10.1007/s10009-015-0380-3>
23. Delahaye, B., Caillaud, B., Legay, A.: Probabilistic contracts: a compositional reasoning methodology for the design of stochastic systems. In: 10th International Conference on Application of Concurrency to System Design, pp. 223–232. IEEE Computer Society (2010). <https://doi.org/10.1109/ACSD.2010.13>
24. Delicaris, J., Schupp, S., Ábrahám, E., Remke, A.: Maximizing reachability probabilities in rectangular automata with random clocks. In: 17th International Sym-

- posium on Theoretical Aspects of Software Engineering, LNCS, vol. 13931, pp. 164–182. Springer (2023). https://doi.org/10.1007/978-3-031-35257-7_10
25. Der Kiureghian, A., Ditlevsen, O.: Aleatory or epistemic? Does it matter? *Struct. Saf.* **31**(2), 105–112 (2009). <https://doi.org/10.1016/j.strusafe.2008.06.020>
 26. Dreossi, T., Dang, T., Donzé, A., Kapinski, J., Jin, X., Deshmukh, J.V.: Efficient guiding strategies for testing of temporal properties of hybrid systems. In: *NASA Formal Methods Symposium*, pp. 127–142. Springer (2015). https://doi.org/10.1007/978-3-319-17524-9_10
 27. Eggers, A., Ramdani, N., Nedialkov, N.S., Fränzle, M.: Improving the SAT modulo ODE approach to hybrid systems analysis by combining different enclosure methods. *Softw. Syst. Model.* **14**(1), 121–148 (2015). <https://doi.org/10.1007/S10270-012-0295-3>
 28. Filliâtre, J.C., Paskevich, A.: Why3 - where programs meet provers. In: *European Symposium on Programming*, pp. 125–128. Springer (2013). https://doi.org/10.1007/978-3-642-37036-6_8
 29. Foster, S., Huerta y Munive, J.J., Gleirscher, M., Struth, G.: Hybrid systems verification with isabelle/hol: simpler syntax, better models, faster proofs. In: *International Symposium on Formal Methods*, pp. 367–386. Springer (2021). https://doi.org/10.1007/978-3-030-90870-6_20
 30. Fränzle, M., Chen, M., Kröger, P.: In memory of Oded Maler: automatic reachability analysis of hybrid-state automata. *ACM SIGLOG News* **6**(1), 19–39 (2019). <https://doi.org/10.1145/3313909.3313913>
 31. Fränzle, M., Kröger, P.: The demon, the gambler, and the engineer – reconciling hybrid-system theory with metrology. In: *Symposium on Real-Time and Hybrid Systems. Theoretical Computer Science and General Issues*, vol. 11180, pp. 165–185. Springer (2018). https://doi.org/10.1007/978-3-030-01461-2_9
 32. Frehse, G., et al.: SpaceEx: scalable verification of hybrid systems. In: *International Conference on Computer Aided Verification*, pp. 379–395. Springer (2011). https://doi.org/10.1007/978-3-642-22110-1_30
 33. Fruth, M.: Probabilistic model checking of contention resolution in the IEEE 802.15.4 low-rate wireless personal area network protocol. In: *Second International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISOLA 2006)*, pp. 290–297. IEEE (2006). <https://doi.org/10.1109/ISoLA.2006.34>
 34. Fulton, N., Mitsch, S., Quesel, J.D., Völz, M., Platzer, A.: KeYmaera X: an axiomatic tactical theorem prover for hybrid systems. In: *International Conference on Automated Deduction*, pp. 527–538, No. 9195 in LNCS, Springer (2015). https://doi.org/10.1007/978-3-319-21401-6_36
 35. Hahn, E., Hartmanns, A., Hermanns, H., Katoen, J.P.: A compositional modelling and analysis framework for stochastic hybrid systems. *Formal Methods Syst. Des.* **43** (10 2013). <https://doi.org/10.1007/s10703-012-0167-z>
 36. Hähnle, R., Huisman, M.: Deductive verification: from pen-and-paper proofs to industrial tools. In: *Computing and Software Science: State of the Art and Perspectives*, LNCS, vol. 10000, pp. 345–373. Springer (2019). https://doi.org/10.1007/978-3-319-91908-9_18
 37. Hartmanns, A., Hermanns, H.: The modest toolset: an integrated environment for quantitative modelling and verification. In: *10th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, LNCS, vol. 8413, pp. 593–598. Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_51

38. Henzinger, T.A., Kopke, P.W., Puri, A., Varaiya, P.: What's decidable about hybrid automata? *J. Comput. Syst. Sci.* **57**(1), 94–124 (1998). <https://doi.org/10.1006/jcss.1998.1581>
39. Herber, P., Fellmuth, J., Glesner, S.: Model Checking SystemC Designs Using Timed Automata. In: *International Conference on HW/SW Codesign and System Synthesis*, pp. 131–136. ACM (2008). <https://doi.org/10.1145/1450135.1450166>
40. Hoare, C.A.R.: Procedures and parameters: an axiomatic approach. In: *Symposium on Semantics of Algorithmic Languages, Lecture Notes in Mathematics*, vol. 188, pp. 102–116. Springer (1971). <https://doi.org/10.1007/BFB0059696>
41. Hoeffding, W.: Probability inequalities for sums of bounded random variables. *J. Am. Stat. Assoc.* **58**(301), 13–30 (1963)
42. Hüllermeier, E., Waegeman, W.: Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Mach. Learn.* **110**(3), 457–506 (2021). <https://doi.org/10.1007/s10994-021-05946-3>
43. Incer, I.: The Algebra of Contracts. Ph.D. thesis, University of California, Berkeley, USA (2022), <https://www.escholarship.org/uc/item/1ts239xv>
44. Kamali, M., Katoen, J.P.: Probabilistic model checking of AODV. In: *Quantitative Evaluation of Systems, LNCS*, vol. 12289, pp. 54–73. Springer (2020). https://doi.org/10.1007/978-3-030-59854-9_6
45. Kong, S., Gao, S., Chen, W., Clarke, E.: dReach: δ -reachability analysis for hybrid systems. In: *Tools and Algorithms for the Construction and Analysis of Systems*, pp. 200–205. Springer (2015). https://doi.org/10.1007/978-3-662-46681-0_15
46. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: verification of probabilistic real-time systems. In: *Computer Aided Verification, LNCS*, vol. 6806, pp. 585–591. Springer (2011). https://doi.org/10.1007/978-3-642-22110-1_47
47. Kwiatkowska, M., Norman, G., Segala, R., Sproston, J.: Automatic verification of real-time systems with discrete probability distributions. *Theoret. Comput. Sci.* **282**, 101–150 (2002). [https://doi.org/10.1016/S0304-3975\(01\)00046-9](https://doi.org/10.1016/S0304-3975(01)00046-9)
48. Legay, A., Delahaye, B., Bensalem, S.: Statistical model checking: an overview. In: *1st International Conference on Runtime Verification, RV 2010, LNCS*, vol. 6418, pp. 122–135. Springer (2010). https://doi.org/10.1007/978-3-642-16612-9_11
49. Li, J., Nuzzo, P., Sangiovanni-Vincentelli, A.L., Xi, Y., Li, D.: Stochastic contracts for cyber-physical system design under probabilistic requirements. In: *Proceedings of the 15th ACM-IEEE International Conference on Formal Methods and Models for System Design*, pp. 5–14. ACM (2017). <https://doi.org/10.1145/3127041.3127045>
50. Liebreinz, T., Herber, P., Glesner, S.: Deductive verification of hybrid control systems modeled in Simulink with KeYmaera X. In: *International Conference on Formal Engineering Methods, LNCS*, vol. 11232, pp. 89–105. Springer (2018). https://doi.org/10.1007/978-3-030-02450-5_6
51. Liebreinz, T., Herber, P., Glesner, S.: A service-oriented approach for decomposing and verifying hybrid system models. In: *International Conference on Formal Aspects of Component Software (FACS), LNCS*, vol. 12018. Springer (2019). https://doi.org/10.1007/978-3-030-40914-2_7
52. Lygeros, J., Prandini, M.: Stochastic hybrid systems: a powerful framework for complex, large scale applications. *Eur. J. Control.* **16**(6), 583–594 (2010). <https://doi.org/10.3166/ejc.16.583-594>
53. MathWorks: Adaptive Cruise Control System. <https://www.mathworks.com/help/reinforcement-learning/examples.html>, online
54. Meyer, B.: Applying ‘design by contract’. *Computer* **25**(10), 40–51 (1992). <https://doi.org/10.1109/2.161279>

55. Mitsch, S., Platzer, A.: ModelPlex: verified runtime validation of verified cyber-physical system models. *Formal Methods Syst. Des.* **49**(1), 33–74 (2016). <https://doi.org/10.1007/s10703-016-0241-z>
56. Mitsch, S., Platzer, A.: The KeYmaera X proof IDE: concepts on usability in hybrid systems theorem proving. In: *Workshop on Formal Integrated Development Environment. Theoretical Computer Science*, vol. 240, pp. 67–81 (2017). <https://doi.org/10.48550/arXiv.1701.08469>
57. Müller, P., Schwerhoff, M., Summers, A.J.: Viper: a verification infrastructure for permission-based reasoning. In: *17th International Conference on Verification, Model Checking, and Abstract Interpretation*, pp. 41–62. Springer (2016). https://doi.org/10.1007/978-3-662-49122-5_2
58. Pilch, C., Edenfeld, F., Remke, A.: Hypeg: statistical model checking for hybrid petri nets: Tool paper. In: *Proceedings of the 11th EAI International Conference on Performance Evaluation Methodologies and Tools*, pp. 186–191 (2017). <https://doi.org/10.1145/3150928.3150956>
59. Platzer, A.: Differential dynamic logic for hybrid systems. *J. Autom. Reason.* **41**(2), 143–189 (2008). <https://doi.org/10.1007/s10817-008-9103-8>
60. Sánchez, C., et al.: A survey of challenges for runtime verification from advanced application domains (Beyond Software). *Formal Methods Syst. Des.* **54**, 279–335 (2019). <https://doi.org/10.1007/s10703-019-00337-w>
61. Schupp, S., et al.: Current challenges in the verification of hybrid systems. In: *Cyber Physical Systems. Design, Modeling, and Evaluation: International Workshop*, pp. 8–24. Springer (2015). https://doi.org/10.1007/978-3-319-25141-7_2
62. da Silva, C., Schupp, S., Remke, A.: Optimizing reachability probabilities for a restricted class of stochastic hybrid automata via flowpipe-construction. *Trans. Model. Comput. Simul.* (2023). <https://doi.org/10.1145/3607197>
63. Strubbe, S.N., Julius, A.A., van der Schaft, A.J.: Communicating piecewise deterministic Markov processes. In: *IFAC Proceedings Volumes* (2003). [https://doi.org/10.1016/S1474-6670\(17\)36449-2](https://doi.org/10.1016/S1474-6670(17)36449-2)
64. Willemsen, L., Remke, A., Ábrahám, E.: Comparing two approaches to include stochasticity in hybrid automata. In: *Proceedings of the 20th International Conference on Quantitative Evaluation of Systems, LNCS*, vol. 14287, pp. 238–254. Springer (2023). https://doi.org/10.1007/978-3-031-43835-6_17
65. Xu, D.N., Gößler, G., Girault, A.: Probabilistic contracts for component-based design. In: *Proceedings of the 8th International Symposium on Automated Technology for Verification and Analysis, LNCS*, vol. 6252, pp. 325–340. Springer (2010). https://doi.org/10.1007/978-3-642-15643-4_24
66. Zhang, J., Zhao, J., Huang, Z., Cao, Z.: Model checking interval probabilistic timed automata. In: *2009 First International Conference on Information Science and Engineering*, pp. 4936–4940. IEEE (2009). <https://doi.org/10.1109/ICISE.2009.749>